

Regex 101

A short introduction to Regular Expressions

Created and maintained by: **Shashank Khobragade**

E-Mail: shashank_dk@hotmail.com

Index

What is RegEx?	2
RegEx Engines	2
Offline and Online RegEx Tester	2
regex101.com	3
RegEx Characters	4
Literal Characters	4
Metacharacters: . + ? ^ \$ - = () { } [] : \	4
Wildcard or Dot Character: .	4
Character Ranges: [a-z] [A-Z] [d-k] [0-9] [3-7]	5
NOT operation character to negate Character Sets and Ranges: ^	6
Escape character: \	6
Escaping Metacharacters in Character Sets	7
Meta Sequences	7
Alternation (Pipe) Character: 	8
Quantifiers: ? * +	9
Limiting Repetitions: { }	10
The Greedy and the Lazy behaviour	11
The Greedy Quantifiers: ? * +	11
The Lazy Quantifiers: ?? *? +?	12
Anchor characters: ^ and \$	14
Word boundary: \b	15
Capturing Groups: ()	15
Backreference for Capturing Groups	16
Non Capturing Groups: (?:)	17
Named Capture Group: (?P<name>)	17
Lookaround Assertions	18
Positive Lookahead Assertion: (?=)	18
Negative Lookahead Assertion: (?!)	19
Positive Lookbehind Assertion: (?<=)	20
Negative Lookbehind Assertion: (?<!)	21

Quizzes and Games	22
Exercises	22
Using RegEx at Work	23
Using RegEx with Text Files and in Programs	24
Linux Commands: grep, sed, awk	24
Bash Scripts	25
Python Programs	25

What is RegEx?

RegEx, or RegExp, the abbreviation for Regular Expressions, is a sequence of characters that specifies a matched pattern in text. They are used by *string-searching-algorithms* to search through a text file, for *find* and *replace* operations, and *input validation*. A RegEx consists of characters from its set of literal characters and metacharacters.

The basic syntax for RegEx is: **/regex/** where regex is the expression to be matched.

RegEx Engines

A RegEx Engine is the tool which processes the RegEx. It is supported by most of the popular programming and scripting languages like Python, Java, JavaScript, Perl (PCRE), Ruby, C++, etc. and applications like Google Sheets and Docs, MS Excel VBA, Notepad++ etc. In the Google environment, tools like Cider and Intent Lab support RegEx enabled searches.

Offline and Online RegEx Tester

Various offline and online tools and websites can be used to test and practice RegEx.

Offline Testers:

Regexo (Google Chrome Extension)

Regex Buddy (paid)

Online Testers:

regex101.com

regexr.com

regexpal.com

RegEx101.com

regex101.com

The default mode/flag set for the RegEx on regex101.com is g: global and m: multiline.
The functions of some of the flags is explained in the table below:

Flag / Mode	Function
/regex/ (no flag)	It matches only the first occurrence of the RegEx in the given text
/regex/g (global)	It matches all the occurrences of the RegEx in the text
/regex/m (multi line)	It returns matches on multiple lines and does not match the newline character. It means it breaks at the newline character and returns matches only for each line.
/regex/s (single line)	The wildcard/dot . character matches newline character.
/regex/i (insensitive)	It ignores the case and matches for both uppercase and lowercase characters.

The other flags can also be controlled through RegEx and can be explored by oneself.

RegEx Characters

Note: Although the basic RegEx syntax requires the format **/regex/** they are added by default on regex101.com. Hence, one can simply write the **regex** while testing.

Notations: For notations sake, the document will make use of **/regex/gm** to specify the regular expression and the active mode/s. **Green** is used to specify the characters that will be included in the match and **red** is used for the ones that are excluded.

(Pay attention to the spaces. The RegEx used are by default **/regex/gm**)

Literal Characters

Literal characters are all the possible Unicode and ASCII characters. They can be used for exact text match searches without specifying any pattern i.e. the text itself is the pattern.

(Please note that the characters coinciding with certain RegEx metacharacters will need to be preceded by the escape character \)

E.g.:

/text/ <https://regex101.com/r/YBvq5k/1>

text textabcdef my_text 123text this is my first text txt

Metacharacters: . + ? ^ \$ | - = () { } [] : \

The metacharacters in RegEx go beyond their meaning as string literals and can be used as operators.

Wildcard or Dot Character: .

The dot character **.** matches any character (including whitespace) other than the newline character (except for the RegEx single line flag: **/regex/s**)

E.g.:

/.at/ <https://regex101.com/r/SAnF3K/1>

cat hat Eat that rotate Create ate at ?at

For `/regex/s` the dot also matches the newline character.

Character Classes / Set: []

Character Class is a set of characters wrapped with square brackets [] and it matches with one of the characters present in it.

E.g.:

`/gr[ae]y/` <https://regex101.com/r/3zvXyf/1>

grey gray prey graey

`/12[345]/` <https://regex101.com/r/2ptuEB/1>

123 124 125 126 12345

`/[Bb]ond-00[07]/` <https://regex101.com/r/KhY2IA/1>

Bond-007 bond-007 Bond-000 bond-000 Bond-0070 Bond007

`/Season[ 1]2/` <https://regex101.com/r/8z66mS/1> (matches either whitespace or 1)

Season12 Season 2 Season 12

Character Ranges: [a-z] [A-Z] [d-k] [0-9] [3-7]

Character ranges returns a match for the entire range between the initial range character and the closing range character. The specified characters must be in the chronological order. For numerals, only the digits from 0 to 9 are allowed in a single range.

[a-z] will match all the lowercase characters from a to z once

[A-Z] will match all the uppercase characters from A to Z once

[0-9] will match all the numerals from 0 to 9 once

[a-zA-Z0-9] will match all the alphanumeric characters once

E.g.:

`/Mercedes Benz [A-Z]-Class/` <https://regex101.com/r/zQSAeR/1>

Mercedes Benz A-Class Mercedes Benz S-Class Mercedes Benz GLE-Class
Mercedes Benz c-Class

`/[a-z][0-9]/` <https://regex101.com/r/fKdlz5/1>

a0 a9 x1 d10 ab20 ab 20 Y01

/regex [A-Za-z3-6]/ <https://regex101.com/r/Ut2Rp6/1> ([3-6] will match numbers from it)

regex A regex 7 regex z regex Aa regex0a regex 3d

/[l-p]/ <https://regex101.com/r/up2nPe/1> (Because the g: global mode is active, the RegEx will return 5 matches between l and p)

abcdefghijklm~~en~~opqrstuvwxyz

NOT operation character to negate Character Sets and Ranges: ^

When the ^ metacharacter precedes a character set or a range, it means the RegEx should exclude it from the match. ^ should be the first character after the opening square bracket [for the negation to work.

(^ can be used to negate only one character at a time. To negate a specific string, one would require the negative assertion operator)

E.g.:

/[^l-p]/ <https://regex101.com/r/n4o4Oc/1> (Because the g: global mode is active, the RegEx will return 24 matches excluding characters from l to p)

abcdefghijklm~~en~~opqrstuvwxyz

/[^5-9]/ <https://regex101.com/r/i8l10S/1> (Matches numbers and alphabets excluding the range between 5 and 9, including the newline character)

2 9 1234 59 49 abcd

/[Bb]ar[^de]/ <https://regex101.com/r/UlvEbl/1>

Bard bare bark Barn

/Do not [^negate] me/ <https://regex101.com/r/1LkNlm/1> (It will not work because ^ negates only one character in [])

Do not negate me Do not - me

Escape character: \

The metacharacters in the RegEx function as operators and hence to specify their literal meaning in the RegEx, certain metacharacters need to be escaped by adding the escape character backslash \ before it.

Any character, except meta-sequence characters, in addition to the metacharacters can be specified as a literal by using the escape character i.e. **a** in the RegEx will be the same as **\a**

Metacharacters that require the escape: **. + ? ^ \$ | () [] **

E.g.:

/2\+2!=5\?/ <https://regex101.com/r/S6wFH6/1> (Escaping the metacharacters + and ?)

2+2!=5? **2\+2!=5\?**

/C:\\Desktop\\Private\\file\\.png/ <https://regex101.com/r/i1LvIS/1> (Escaping the metacharacters \ and .)

C:\Desktop\Private\file.png

C:\\Desktop\\Private\\file\\.png

Escaping Metacharacters in Character Sets

When the RegEx contains the following metacharacters in a character set, they might at times need to be escaped: **- /]**

Whereas the other metacharacters will carry their literal meaning within the character set.

E.g.:

/[.\\V]/ <https://regex101.com/r/qrFPm0/1> (Escaping] and /)

[.] **[.]** **[.]**

Meta Sequences

A few literal characters preceded by the escape character can be used to represent character ranges as well as RegEx operators.

Meta Sequence	Equivalent character/range	Example
\s	Whitespace character for space, tab or newline	this\sand\s that https://regex101.com/r/MFmand/1 this and that this\sand\s that
\S	Non-whitespace character	\s\S\S https://regex101.com/r/nL7OAt/1

		a (two whitespaces and one non-whitespace) (three whitespaces not matched)
\d	Any digit [0-9]	\d-\d https://regex101.com/r/TAD243/1 3-2 a-1
\D	Any non-digit [^0-9]	\D:\D https://regex101.com/r/yIkVJ2/1 22:7 a:b +.:?
\w	Any word character and underscore [a-zA-Z0-9_]	\w\w\w https://regex101.com/r/k5hdOW/1 A_3 a+3
\W	Any non-word character	\W\W\W https://regex101.com/r/xEVPx/1 (? won't be highlighted, as it also matches newline for the first \W) A_3 !+?
These are the most frequently used Meta Sequences for basic RegEx matching. There are several others and can be found under Meta Sequences section in Quick References on regex101.com		

Alternation (Pipe) Character: |

The alternation character denoted by the metacharacter pipe | can provide choices within a RegEx. To get error-free matches, one should provide the choices wrapped in round brackets ()

E.g.:

/dog|cat/ <https://regex101.com/r/LktH4L/1>

I am a dog person

I am a cat person

I am a dog and cat person

/I am a dog|cat person/ <https://regex101.com/r/MxePoe/1> (This doesn't give desired match, as the choice is not between **dog** and **cat**, but between **I am dog** and **cat person**)

I am a dog person

I am a cat person

I am a dog and cat person

/I am a (dog|cat) person/ <https://regex101.com/r/ayOH6K/1> (Round brackets around dog|cat fixes the match)

I am a dog person

I am a cat person

I am a dog and cat person

Quantifiers: ? * +

RegEx provides the three metacharacters **?** ***** and **+** as quantifiers to match repetitions of the previous character or token. The three quantifiers differ in the number of repetitions to be matched.

Quantifier	Repetition number	Examples
?	Matches zero or one occurrence of the previous token/character. It can be used to specify a token as optional.	/RegExp?/ https://regex101.com/r/Op7cSt/1 (p may occur either 0 or 1 time, hence optional) RegEx RegExp
*	Matches zero or more occurrences of the previous token/character	/\d* m/s/ https://regex101.com/r/WKmZ48/1 (0 or more occurrence of a digit) 299792458 m/s

		<code>/[a-zA-Z.]*/</code> https://regex101.com/r/OSqZfY/1 (Repeats the character set for lowercase, uppercase, whitespace and period 0 or more times) "Carpe diem. Seize the day."
+	Matches one or more occurrence of the previous token/character	<code>/\d+[A-Za-z]+/</code> https://regex101.com/r/EyZBSx/1 (Matches a digit 1 or more times followed by the repeating character set 1 or more times) 273K <code>/[A-Z]+\d*/</code> https://regex101.com/r/YQA1u4/1 ([A-Z] must occur at least once, whereas \d may occur zero or more times after [A-Z]) K129 CLOUD9 OK 989

Limiting Repetitions: { }

The repetitions of previous token/character can be controlled by specifying either **the maximum or the minimum number of repetition**, or **the range for repetition (at least - up to)**. These values need to be specified wrapped with curly brackets.

Repetition Notation	Explanation	Examples
{n}	Repeat the previous token/character n number of times	<code>/\d{2}\.\d{2}\.\d{4}/</code> https://regex101.com/r/Jn4JRp/1 (Matches date dd.mm.yyyy: \d repeated 2, 2 and 4 times) 05.04.2024 5.4.2024

		/[A-Z]{1}\d{2} [A-Z]{2}\d{4}[A-Z]{2}/ https://regex101.com/r/QapyO8/1 (Matches Eircode format in Ireland. There's a space between the first 3 and the next 4 characters.) D03 XE64 W91 AK1Y D04 A55
{n,m}	Repeat the previous token/character at least n and at the most m number of times	/\w{15,33}/ https://regex101.com/r/MzhJWI/1 (Validates IBAN length of characters: Minimum 15 for Norway and 33 for Russia) RU0204452560040702810412345678901 (Russia: 33) NO8330001234567 (Norway: 15) IN0123456789 (Invalid: 12) RUSSIA0204452560040702810412345678901 (Invalid: 37)
{n,}	Repeat the previous token/character at least n number of times	/[!@#%&'()*+,-\;:;<=>?`~]{8,}/ https://regex101.com/r/JK0dH0/1 (Password must be at least 8 characters long) shdhAA187+_ pass123 hdbcaskjd323?

The Greedy and the Lazy behaviour

The Greedy Quantifiers: ? * +

The quantifiers used for repetition namely **? * +** behave by default in the so-called **Greedy** manner. They try to match as many characters as possible, giving back as needed, before transferring the control to the next RegEx character/token, if present.

[/a.+a/](https://regex101.com/r/4aCjBu/1) <https://regex101.com/r/4aCjBu/1>

greedy can be dangerous at times

In the above example, the RegEx looks for the first **a**, moves on to the next character i.e. the **.** dot wildcard character. It matches the next character **n** and the RegEx now sees the quantifier **+** and realizes that the wildcard should repeat itself until it finds the next token i.e. **a**. So we expect the RegEx to match the string **greedy can be dangerous at times** because the next **a** occurs much earlier than in the example. However, we are proven otherwise and instead the last occurring **a** is matched.

This happens due to the **greedy** nature of the repeating quantifiers.

The stepwise greedy matching algorithm is explained below:

/a.+a/ <https://regex101.com/r/4aCjBu/1>

greedy can be dangerous at times

Step 1:

The first RegEx character is the literal /**a**.+a/ which is matched at position 8, while the previous 7 characters are skipped due to no-match:

greedy can be dangerous at times

Step 2:

The second RegEx character is the wildcard character /**a**.+a/ which is matched at position 9:

greedy can be dangerous at times

Step 3:

The next RegEx character is the quantifier + (matches previous character 1 or more times) /**a**.+a/ which due to its greedy nature tries to consume all the available characters till the end of the string:

greedy can be dangerous at times

Step 4:

The control is now transferred to the next RegEx character **a** /**a**.+a/ However, it cannot find its match because there are no more characters available. So our previous greedy quantifier gives some of its consumed characters back (giving back) to **a**. So the character **a** now turns back (backtracks) and tries to match with the first instance when it finds it, in this case at position 25:

greedy can be dangerous at times

Thus we get the above match.

Example 2:

/"."+"/ <https://regex101.com/r/bX3wRP/1> (While trying to match the phrases occurring between the quotations, we get the following match due to the greedy *)

Hamlet: "To match" or "not to match", or rather "what to match", that is the question.

The Lazy Quantifiers: ?? *? +?

The counter behaviour to overcome the greedy nature of the quantifiers is the **Lazy** approach. The lazy quantifiers try to match the previous character as few times as possible, expanding as needed. The greedy quantifiers can be coerced into behaving lazy by adding the **?** character after them.

The above examples will return desired matches by making the quantifiers lazy.

Example 1:

/a.*?a/ <https://regex101.com/r/AYr13f/1> (? after * makes it lazy)

greedy can be dangerous at times

Example 2:

/"."+?"/ <https://regex101.com/r/bX3wRP/1> (? after + makes it lazy)

Hamlet: "To match" or "not to match", or rather "what to match", that is the question.

The stepwise lazy matching algorithm is explained below:

/a.+?a/ <https://regex101.com/r/4aCjBu/1>

greedy can be dangerous at times

Step 1:

The first RegEx character is the literal /**a**.+?a/ which is matched at position 8, while the previous 7 characters are skipped due to no-match:

greedy can be dangerous at times

Step 2:

The second RegEx character is the wildcard character /**a**.+?a/ which is matched at position 9:

greedy can be dangerous at times

Step 3:

The next RegEx character is the quantifier + (matches previous character 1 or more times) `/a.+?a/` which due to its lazy nature has already done the bare minimum i.e. by matching `n`. It now transfers the RegEx control to the next character i.e. `a` at the end `/a.+?a/` which returns a negative match because it is a whitespace character: `greedy can` be dangerous at times

Step 4:

Because `a` could not find a match, the control goes back to the quantifier and it expands character by character as needed. This back and forth continues until `a` finds a positive match at position 16 to give us the desired result.

`/a.+?a/`
`greedy can` be dangerous at times
`/a.+?a/`
`greedy can` be dangerous at times
`/a.+?a/`
`greedy can` be dangerous at times
`/a.+?a/`
`greedy can` be dangerous at times

The lazy approach matches only the bare minimum, if it is not required to expand: `/\d+ \d+?/` <https://regex101.com/r/tmyK5v/1> (The first `\d+` is greedy and matches every digit until the whitespace character, while the second `\d+?` is lazy and matches only one instance of a digit, as few characters as possible, because it is not being asked to expand.)

`123 456`

Anchor characters: ^ and \$

The purpose of the anchor characters `^` and `$` is only to specify the beginning and the end of a line respectively. Neither of the characters matches with a string, but are used only as placeholders to denote the start and end of a line.

Anchor Character	Explanation	Examples
<code>^</code>	It denotes the beginning of a line. The RegEx character/set/range	<code>/^[A-Z]\w+/</code> https://regex101.com/r/CjcFou/1 (The line must begin with an uppercase

	following it will return a positive match <i>only if</i> it is at the beginning of the line.	letter) RegEx is amazing regex is cool REGEX is <3 <code>/^[A-Z]{2} \d{1,4}/</code> https://regex101.com/r/EgTxVZ/1 (Validates flight number, must start with two uppercase characters) LH 982 BA 8737 AF 1819 AIN 123 LHE 920
\$	It denotes the end of a line. The RegEx will match the text only if it matches with the RegEx preceding the \$ character with the string at end of the line.	<code>/.+ment\$/</code> https://regex101.com/r/Yunq4f/1 (Lines ending with <i>ment</i> will be matched) apartment seize the moment meant
^ \$	The combination of ^ and \$ can be used to specify both the beginning and the end of the line i.e. each line is now bounded	<code>/^Humpty.*wall\$/</code> https://regex101.com/r/tScBWF/1 (only the first line of the nursery rhyme will match) Humpty Dumpty sat on a wall Humpty Dumpty had a great fall

Word boundary: \b

The RegEx character **\b** sets a word boundary immediately between a character matched by **\w** and not matched by **\w**. It is another example of an anchor in RegEx which just specifies the boundary, in this case of a word and does not match any characters: <https://regex101.com/r/CWbfnH/1>

E.g.:

`/s\b/` <https://regex101.com/r/XapYaj/1> (**\b** sets the boundary after **s** matches with **s**)

This is a string of some words
We are binding these words

`/\b\w+\b/` <https://regex101.com/r/z7Cjgk/1> (`\b` sets a boundary around all the words, while `\w+` matches the word)

This is a string of some words
We are binding these words

`/i\b/` <https://regex101.com/r/Wlwlgu/1> (This won't match any character, because there is no word ending in `i`)

This is a string of some words
We are binding these words

Capturing Groups: ()

A RegEx wrapped with round brackets not only captures the entire match for the (sub-)RegEx between the brackets, but also saves this match that can be reused further in the RegEx. This captured match is called a **Capturing Group**, and is auto-assigned with a numerical order, which means if there are more than one capturing groups in a RegEx, then the first will get the number 1, second 2, and so on.

`/This is our ([\w ]+), our ([\w ]+) and our ([\w ]+)\./` <https://regex101.com/r/ieS9nh/1>
This is our 1st Capturing Group, our 2nd Capturing Group and our 3rd Capturing Group.

`/.+(enie|meeny|miny|moe)/` <https://regex101.com/r/4TvbYE/1> (The only Capturing Group here is the alternation match wrapped between the brackets. Each line matches one of the four options.)

Whom do I choose? enie
Or should I choose? meeny
Or maybe? miny
Or rather? moe

Backreference for Capturing Groups

It was mentioned above that the Capturing Groups can be reused further within the RegEx because the Capturing Groups save the matched items i.e. capture them. This

reuse is done through **Backreference**. The syntax for backreferencing is the escape character followed by the capturing group number. Therefore, for backreferencing the **1st** capturing group the syntax will be **\1**, for the **2nd** capturing group it will be **\2**, for **10th** it will be **\10**, and so on.

/I am a (dog|cat) person\.. I love \1s\./ <https://regex101.com/r/lkbz7j/1> (The Capturing Group matches a with either *dog* or *cat* and the backreference \1 reuses this match. In the first two lines below, the Capturing Group captures **dog** and **cat** respectively, and when it sees **dog** and **cat** again in the corresponding line, \1 asks the RegEx to match it again. Whereas in the third and fourth line, it captures **cat** and **dog** respectively, but the corresponding lines do not contain the same string again, and hence we get no match.)

I am a dog person. I love dogs.

I am a cat person. I love cats.

I am a cat person. I love dogs. (cat is captured, but later we have dogs)

I am a dog person. I love cats. (dog is captured, but later we have cats)

/I am (\w+).*\1/ <https://regex101.com/r/BA8RNI/1>

I am Bond, James Bond

I am Max, Mad Max

I am Wayne, Bruce Wayne

I am potato, couch potato

/John: (\d+) (\w+)\nPaula: (\d+) \2\nDaniel: (\d+) \2\n(.\+3.\+1.\+4.\+)/

<https://regex101.com/r/T8VIOt/1> (4 Capturing groups and 4 Backreferences)

John: 200 points matched by John: (\d+) (\w+), 200 and points captured

Paula: 250 points matched by Paula: (\d+) \2, 250 captured points backreferenced

Daniel: 175 points matched by Daniel: (\d+) \2, 175 captured points backreferenced

Paula got the highest points 250, followed by John with 200 points and Daniel 175 points. matched by (.\+3.\+1.\+4.\+) and the sentence is captured too

Non Capturing Groups: (?:)

The Capturing Groups always save the captured match and thus use up memory resulting in a slowed down execution. It may not be an issue in small text files or for

small programs. But for large files and data, having many Capturing Groups may make the matching process much longer than required. The **Non Capturing Group** provides a solution to this by allowing us to declare a Capturing Group as Non Capturing by simply adding **?:** right after the opening round bracket. This will assert to the RegEx that this group must not be captured i.e. saved. Unless you are using a Capturing Group by Backreferencing, it is always a good practice to declare the group as Non Capturing.

/I am a (?:dog|cat) person. I love \1s\./ (The (?:dog|cat) group is now a Non Capturing Group, and hence cannot be backreferenced using \1. The text will return no match)

I am a dog person. I love dogs.

/.*(?:Ireland|Scotland).*(rainy|cloudy).*\1.*/ <https://regex101.com/r/AD01uk/1> (The first group is Non Capturing whereas the second one is the only Capturing Group and hence can be backreferenced using \1. The Non Capturing Group can still be used for alternation and will match with either Ireland or Scotland.)

In Ireland it is always rainy. I don't like rainy weather.

In Scotland it is always rainy. I don't like rainy weather.

In Ireland it is always cloudy. I don't like cloudy skies.

In Ireland it is always rainy. I don't like cloudy skies.

Named Capture Group: (?P<name>)

The Capturing Groups can also be captured using a custom name. This name can be later used for Backreferencing. Named Capture Group follow the syntax **(?P<name>)** while Backreferencing uses the syntax **(?P=name)**

/.+?(?P<num>\d+)\. (?P=num).+ / <https://regex101.com/r/YG8GMI/1> (The group (\d+) is now captured using the name **num** and is backreferenced using the same name)

Can I have your number? Sure, it is 9876543210. 9876543210, is that correct?

/John: (?P<john>\d+) (?P<point>\w+)\nPaula: (?P<paula>\d+) (?P=point)\nDaniel: (?P<daniel>\d+) (?P=point)\n(.+?(?P=paula).+?(?P=john).+?(?P=daniel).+?) /

<https://regex101.com/r/kxMmwa/1> (Same example from [here](#) using Named Groups)

John: 200 points

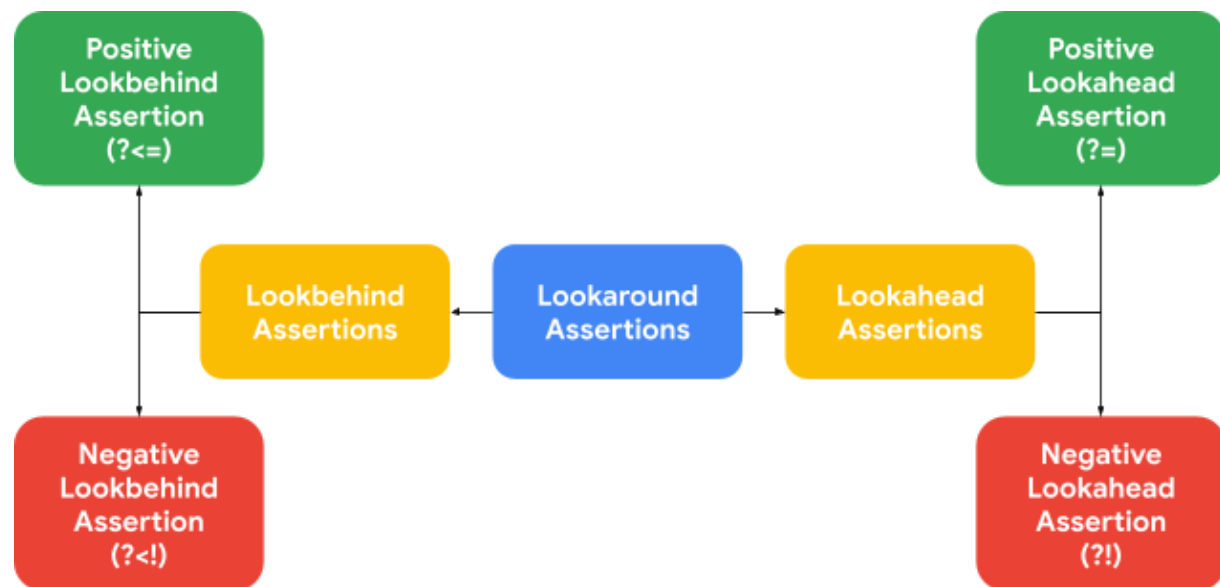
Paula: 250 points

Daniel: 175 points

Paula got the highest points 250, followed by John with 200 points and Daniel 175 points.

Lookaround Assertions

The Lookaround Assertions in RegEx provide the possibility to return a positive match when a certain condition is met within the RegEx. It does so when this condition is another Sub-RegEx specified in the Lookaround syntax either to the right or left of the position where it is asserted. Based on this direction, the Lookaround Assertions are classified into two main types and each type has two categories i.e. positive or negative.



Positive Lookahead Assertion: (?=)

When the assertion asks the RegEx to return a match if a positive match lies ahead of it i.e. to its right in the line. The syntax for the positive lookahead is: **(?=)** and it should be placed in the RegEx at the same position where this string/character is located.

/Rock (=?Scissors)/ <https://regex101.com/r/fcVUk2/1> (In the two lines below, *Rock* will be matched only when *Scissors* follows it. Please also see that *Scissors* won't be matched because it is used only for the assertion and is not part of the RegEx.)

Rock Scissors

Rock Paper

/Rock (?=Scissors)\w+/ <https://regex101.com/r/xigJ10/1> (To include Scissors in the match, it needs to be specified as done with **\w+** in this case)

Rock Scissors

Rock Paper

There can also be multiple assertions within a single RegEx:

/(.+: e-mail: (?=\w+@\w+\.\w+)([\w_]+@[\w- _]+\.\w+), mobile: (?=\d{10})\d{10})/ <https://regex101.com/r/MF6VAB/1> (The first positive lookahead asserts that only if the following string after the colon and whitespace is of the email address pattern **(?=\w+@\w+\.\w+)** AND the mobile number pattern with 10 digits **(?=\d{10})**, only then match not just the entire line, but also capture the phrase *The data is correct*, the email address and the phone number using Capturing Groups.

The data is correct: e-mail: notmyemail@gibberish.com, mobile: 1234567890

The data is correct: e-mail: notmyemail@gibberishcom, mobile: 1234567890

(Incorrect domain name, does not match the assertion **(?=\w+@\w+\.\w+)**)

The data is correct: e-mail: notmyemailgibberish.com, mobile: 1234567890

(Incorrect domain name, does not match the assertion **(?=\w+@\w+\.\w+)**)

The data is correct: e-mail: notmyemail@gibberish.com, mobile: 1234567 (Incorrect mobile number, does not match the assertion **(?=\d{10})**)

Negative Lookahead Assertion: (?!)

The positive assertion returns a RegEx match when the sub-RegEx is present ahead, whereas the **Negative Lookahead Assertion** works when the sub-RegEx is **not** present ahead.

/Rock (?!Paper)\w+/ (If we replace Scissors in the above example and assert using the negative lookahead method, that if the string following Rock is not Paper, then the RegEx should match the entire line i.e. Rock wins)

Rock Scissors

Rock Paper

`/.+Fuel: (?!Diesel).+Transmission: (?!Automatic)\w+/?`

<https://regex101.com/r/lyztEDF/1> (From the list of cars, the RegEx will filter out the options for *Fuel: Diesel* and *Transmission: Automatic*. It is using the negative lookahead assertion to return the match that does not contain these two matches)

Make: BMW, Model: i118, Fuel: Petrol, Transmission: Automatic

Make: Volkswagen, Model: Polo, Fuel: Petrol, Transmission: Manual

Make: Audi, Model: A4, Fuel: Diesel, Transmission: Manual

Make: BMW, Model: i116, Fuel: Diesel, Transmission: Automatic

Positive Lookbehind Assertion: `(?<=)`

The **Positive Lookbehind Assertion** functions in the same way as the positive lookahead assertion. The only point of differentiation between these two is that the latter looks ahead in the RegEx and returns a match when the condition is satisfied, whereas the **Positive Lookbehind Assertion** looks behind to its left in each line. Its syntax follows the notation `(?<=)`.

`/(?<=Sun)day/` <https://regex101.com/r/KB54cs/1> (The RegEx matches *day* when the assertion looks behind and finds **Sun** as a match)

Monday

Tuesday

Wednesday

Thursday

Friday

Saturday

Sunday

`/.(?<=Sun)day/` <https://regex101.com/r/WWGYZZ/1> (The RegEx matches the entire line instead of only *day*)

Monday

Tuesday

Wednesday

Thursday

Friday

Saturday

Sunday

`/.+(?<=tion)/` <https://regex101.com/r/8aef2D/1> (The RegEx looks behind from the end of the line to see if the line contains **tion** in it and matches all the characters in only those lines for the *Californication* lyrics)

Psychic spies from China try to steal your mind's elation
And little girls from Sweden dream of silver screen quotation
And if you want these kind of dreams it's Californication
It's the edge of the world and all of Western civilization
The sun may rise in the East, at least it settled in a final location
It's understood that Hollywood sells Californication
Pay your surgeon very well to break the spell of aging
Celebrity skin, is this your chin, or is that war you're waging?

The same can be achieved using positive lookahead assertion: `/.+(?=tion).*/`
<https://regex101.com/r/G7aUUA/1>

Negative Lookbehind Assertion: `(?<!)`

Just like the negative lookahead assertion behaves as the counterpart for positive lookahead, the **Negative Lookbehind Assertion** behaves in the opposite way to the Positive Lookbehind Assertion. The syntax is denoted by `(?<!)`.

`\w+(?<!Sun)day/` <https://regex101.com/r/Wq7Q9g/1> (The RegEx matches the `\w+` and the remaining characters until day and then looks behind from the position and returns all those matches that don't contain **Sun**)

Monday
Tuesday
Wednesday
Thursday
Friday
Saturday
Sunday

`/.+(?<!tion)?/` <https://regex101.com/r/JNgffG/1> (For the *Californication* lyrics example, the negative lookbehind assertion will look behind from the end of the line and match all the ones that don't contain **tion** at the end.)

Psychic spies from China try to steal your mind's elation

And little girls from Sweden dream of silver screen quotation
And if you want these kind of dreams it's Californication
It's the edge of the world and all of Western civilization
The sun may rise in the East, at least it settled in a final location
It's understood that Hollywood sells Californication
Pay your surgeon very well to break the spell of aging
Celebrity skin, is this your chin, or is that war you're waging?

Quizzes and Games

Activity	Link
Revise the RegEx Metacharacters to play this game	https://wordwall.net/play/71166/791/460
Revise the Meta Sequences for this game	https://wordwall.net/play/71167/557/655
Revise the different groups of RegEx characters, sequences and quantifiers	https://wordwall.net/play/71167/989/414
Reorder the RegEx in the correct sequence	https://learningapps.org/watch?v=p9yecq11j24
Match the terms with the syntax	https://wordwall.net/play/71842/308/472
Crossword: Find the correct RegEx match	https://wordwall.net/play/71845/824/372

Exercises

Title	Description
Improved IBAN character length validator	https://www.iban.com/structure The first two characters are uppercase

	for the country code, followed by between 13 and 31 alphanumeric characters, where the alphabets are in uppercase Possible solution: https://regex101.com/r/Wtaap7/1
Matching phrases	Use alternation method to match the following phrases: BMW is the best car in the world BMW is the worst car in the world Audi is the best car in Europe BMW is the best car in Europe Audi is the worst car in Europe Possible solution: https://regex101.com/r/voaSlP/1
Improved password validator	- The minimum password length should be 10 and maximum 15 characters - The password should not begin with a digit or a special character - It should contain at least one special character: ~`!@#\$%^&*()_-=+{[]\ :;'"<,>./ - It should contain at least one uppercase character - Spaces are not allowed
